

Session 4 · Deck 4.B1

Function Approximation and Interpolation

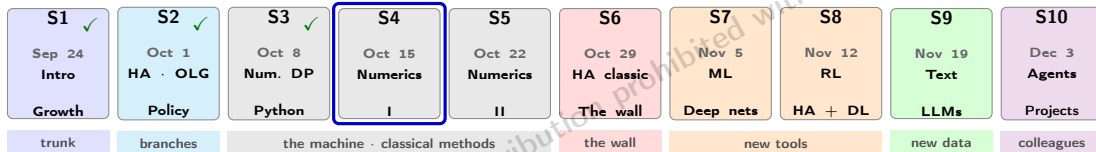
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.B1 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 4.B1 approximation and interpolation · 4.B2 expectations: quadrature and the Markov chain · 4.B3 fast and accurate · 4.B4 the worked example, end to end.

More Accurate, and Wrong

Interpolate the policy $g(k) = \min(k, 4)$ — a household that saves everything until a constraint binds — through 21 nodes, three ways:

	max error	overshoots?
linear	0	no
cubic spline	0.043	yes
PCHIP	0.074	no

The cubic spline — the one with $O(h^4)$ accuracy — puts the policy **above a cap the model says cannot be exceeded**, and loses both monotonicity and concavity doing it.

Why this is not a small thing

Feed that interpolant back into a Bellman loop and the maximization is no longer unimodal — so 4.A1's golden section is no longer licensed, and it can converge to the wrong point.

Accuracy order is not the criterion. **The criterion is which properties survive**, because the algorithms downstream depend on them.

Why a Dynamic Program Needs This at All

$\hat{v}$ is known at N grid points and nowhere else. But:

- 4.A1's golden section asks for $\hat{v}(k')$ at points it chooses, which are not grid points;
- time iteration evaluates c_n at $k' = y - c$, which is not a grid point;
- simulation visits states that are not grid points;
- Euler errors are deliberately evaluated *off* the grid.

And the error does not stay put: an interpolation error enters $\hat{v}$, which enters the next sweep, which enters the next. **It compounds through the fixed point.**

Two questions, always

Local or global? Local schemes (linear, splines) use nearby nodes; global ones (polynomials, Chebyshev) use all of them, so one bad node moves the fit everywhere.

What survives? Monotonicity, concavity, and the bounds the economics imposes.

Linear: The One That Keeps Its Promises

Between k_i and k_{i+1} ,

$$\hat{v}(k) = v_i + \frac{v_{i+1} - v_i}{k_{i+1} - k_i}(k - k_i),$$

with error $O(h^2)$ and a derivative that jumps at every node.

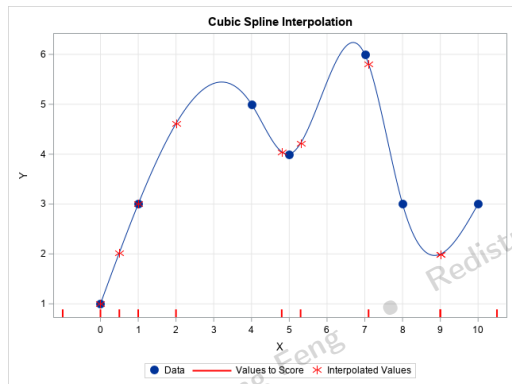
$\ln k$, 12 nodes	monotone	concave
linear	yes	yes
cubic spline	yes	yes
PCHIP	yes	yes

Read the concavity row

Linear interpolation **does** preserve concavity: the chord of a concave function lies below it, so the piecewise-linear interpolant is concave — worst second difference here, 4×10^{-16} .

It matters because the golden-section argument of 4.A1 needs a concave $\hat{v}$, and **linear interpolation is what supplies it** — the humblest scheme on the table guarantees the property the method depends on.

Cubic Splines: Smooth, Accurate, and Prone to Overshoot



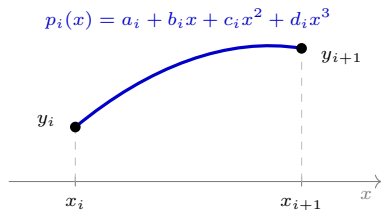
Cubic spline through sparse nodes.

A cubic on each interval, matched in value, slope and curvature at every node: C^2 overall, error $O(h^4)$.

The smoothest interpolant. Among all twice-differentiable functions through the data, the natural cubic spline has the least total curvature $\int (s'')^2 dx$ — the shape a flexible drafting strip takes when pinned at the nodes, hence the name.

The same smoothness that buys $O(h^4)$ is what makes it overshoot near a kink: the spline cannot turn sharply, so it swings past instead.

Piecewise Cubics: $4(N - 1)$ Unknowns Against the Equations



one cubic per interval: four coefficients to find

N nodes, $N - 1$ intervals, a cubic on each. Nothing else is assumed about the shape; the equations do the rest.

Unknowns: $4(N - 1)$ coefficients. **Equations we insist on:**

interpolate: $p_i(x_i) = y_i$ and $p_i(x_{i+1}) = y_{i+1}$, each $2(N - 1)$
piece

slopes agree at each interior node: $p'_{i-1}(x_i) = p'_i(x_i)$ $N - 2$

curvatures agree at each interior node: $N - 2$

$p''_{i-1}(x_i) = p''_i(x_i)$

total $4N - 6$

short by **2**

The two missing equations are the **boundary condition**, your choice: *natural* ($p'' = 0$ at both ends), *clamped* (p' given there), or *not-a-knot*. Then $4(N - 1)$ equations for $4(N - 1)$ unknowns: written out on the next frame.

PCHIP, by contrast

Same cubic per interval, no curvature equations: it fixes each node's *slope* from the data; a piece's four coefficients then follow from its two end values and two end slopes.

The Equations, Written Out

On interval i : $p_i(x) = a_i + b_i x + c_i x^2 + d_i x^3$, so

$$p'_i(x) = b_i + 2c_i x + 3d_i x^2, \quad p''_i(x) = 2c_i + 6d_i x.$$

Interpolation, both ends of every piece:

$$a_i + b_i x_i + c_i x_i^2 + d_i x_i^3 = y_i,$$

$$a_i + b_i x_{i+1} + c_i x_{i+1}^2 + d_i x_{i+1}^3 = y_{i+1}.$$

Interior node x_i , where piece $i-1$ ends and piece i starts — same slope, same curvature:

$$b_{i-1} + 2c_{i-1}x_i + 3d_{i-1}x_i^2 = b_i + 2c_i x_i + 3d_i x_i^2,$$

$$2c_{i-1} + 6d_{i-1}x_i = 2c_i + 6d_i x_i.$$

Boundary nodes x_1, x_N : one piece touches each, nothing to match. *Natural* sets that piece's curvature to zero there:

$$2c_1 + 6d_1 x_1 = 0, \quad 2c_{N-1} + 6d_{N-1} x_N = 0.$$

Clamped instead: $p'_1(x_1)$ and $p'_{N-1}(x_N)$ set to given slopes. Codes shift the origin to each node, $x \rightarrow x - x_i$, for stability; the equations are the same.

Ledger `spline_power_form_3pt`.

Example: nodes (3.5, 3.5), (4, 4), (4.5, 4)

$$a_1 + 3.5b_1 + 12.25c_1 + 42.875d_1 = 3.5$$

$$a_1 + 4b_1 + 16c_1 + 64d_1 = 4$$

$$a_2 + 4b_2 + 16c_2 + 64d_2 = 4$$

$$a_2 + 4.5b_2 + 20.25c_2 + 91.125d_2 = 4$$

$$b_1 + 8c_1 + 48d_1 - b_2 - 8c_2 - 48d_2 = 0$$

$$2c_1 + 24d_1 - 2c_2 - 24d_2 = 0$$

$$2c_1 + 21d_1 = 0$$

$$2c_2 + 27d_2 = 0$$

Rows 1–4 interpolate (p_1 on $[3.5, 4]$, p_2 on $[4, 4.5]$); 5 slopes agree at 4; 6 curvatures agree at 4; 7–8 zero curvature at 3.5 and 4.5. Eight equations, eight coefficients.

Solving It: One Banded System, and Why a Spline Is Global

	a_1	b_1	c_1	d_1	a_2	b_2	c_2	d_2
interpolate	•	•	•	•				
interpolate	•	•	•	•				
interpolate					•	•	•	•
interpolate					•	•	•	•
slopes agree		•	•	•		•	•	•
curvatures agree			•	•			•	•
natural end			•	•				
natural end						•	•	

The three-node system's nonzeros. Each equation touches one piece, or two neighbours: **banded**, so elimination costs $O(N)$ however many nodes. The boundary rows (red) each touch one piece.

The pieces are chained: the slope and curvature equations link piece i to piece $i - 1$, which is linked to $i - 2$, and so on to the boundary. So **one data value moves every coefficient**, and a boundary condition is felt everywhere. On the 21-node $\min(k, 4)$ data the solved slopes $p'(x_i)$ near the kink:

node k	δ left	δ right	slope p'
3.5	1	1	1.13
4.0	1	0	0.50
4.5	0	0	-0.13
5.0	0	0	0.04

Coupled, hence global: at the kink the data flatten, but the cubic must keep its curvature continuous, so the slopes **ring** — 1.13 just before the cap, -0.13 just after, shrinking by about 3.7 per node: the overshoot of the first frame.

Codes first eliminate a_i, b_i, c_i, d_i down to the N node slopes, a tridiagonal system (de Boor 1978). Ledger `spline_slope_system`, `spline_power_form_3pt`.

PCHIP: The Same Cubics, Different Equations

Symbols

$h_i = x_{i+1} - x_i$, the length of piece i (one number h on an even grid; 0.5 below). $\delta_i = (y_{i+1} - y_i)/h_i$, the **secant**, the data's slope over piece i . m_i , the slope we **prescribe** at node i .

Same cubic on each interval, $p_i(x) = a_i + b_i x + c_i x^2 + d_i x^3$, and the same $2(N - 1)$ interpolation equations $p_i(x_i) = y_i$, $p_i(x_{i+1}) = y_{i+1}$. **What changes.** The spline's other $2(N - 1)$ equations (slopes, curvatures, boundary) are replaced by **prescribing the slope at every node**, m_i , chosen from the data (next frame): for each piece $p'_i(x_i) = m_i$ and $p'_i(x_{i+1}) = m_{i+1}$, another $2(N - 1)$, total $4(N - 1)$ again. **Why it is local.** A piece's four equations involve only its own four coefficients: one 4×4 solve per piece. Both pieces at a node get the same m_i , so slopes agree there.

Three nodes again

Nodes (3.5, 3.5), (4, 4), (4.5, 4), $h = 0.5$; secants $\delta_1 = 1$, $\delta_2 = 0$; the rule (next frame) gives $m_1 = 1.5$, $m_2 = 0$, $m_3 = 0$:

$$a_1 + 3.5b_1 + 12.25c_1 + 42.875d_1 = 3.5$$

$$a_1 + 4b_1 + 16c_1 + 64d_1 = 4$$

$$b_1 + 7c_1 + 36.75d_1 = 1.5$$

$$b_1 + 8c_1 + 48d_1 = 0$$

$$a_2 + 4b_2 + 16c_2 + 64d_2 = 4$$

$$a_2 + 4.5b_2 + 20.25c_2 + 91.125d_2 = 4$$

$$b_2 + 8c_2 + 48d_2 = 0$$

$$b_2 + 9c_2 + 60.75d_2 = 0$$

Rows 1–2, 5–6 interpolate, as for the spline; 3–4, 7–8 prescribe the slopes. No unknown is shared: two separate 4×4 systems. Piece 2, value 4 and slope 0 at both ends, is the flat line: no overshoot.

Curvature is not matched: C^1 , not C^2 . Fritsch & Carlson (1980). Ledger pchip_power_form_3pt: the two 4×4 solves reproduce SciPy's PchipInterpolator.

PCHIP: How the Node Slopes Are Chosen From the Data

Interior node, with the two secants δ_{i-1} , δ_i on either side:

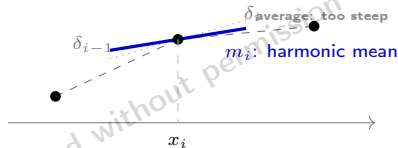
$$m_i = \frac{w_1 + w_2}{w_1/\delta_{i-1} + w_2/\delta_i} \text{ if } \delta_{i-1}\delta_i > 0, \quad m_i = 0 \text{ else,}$$

$w_1 = 2h_i + h_{i-1}$, $w_2 = h_i + 2h_{i-1}$: a weighted **harmonic mean** of the two secants, leaning toward the flatter one; on an even grid $w_1 = w_2$ and $m_i = 2/(1/\delta_{i-1} + 1/\delta_i)$. Where the secants differ in sign or one is zero, the data turn, and $m_i = 0$.

End node, one neighbour only: the slope at x_1 of the parabola through the first three points,

$$m_1 = \frac{(2h_1 + h_2)\delta_1 - h_1\delta_2}{h_1 + h_2} \quad (\text{even grid: } \frac{3\delta_1 - \delta_2}{2}),$$

guarded: $m_1 = 0$ if it points against δ_1 ; $m_1 = 3\delta_1$ if δ_1, δ_2 differ in sign and $|m_1| > 3|\delta_1|$. Mirror at x_N . **No boundary condition to guess.**



Worked

Three nodes (3.5, 3.5), (4, 4), (4.5, 4), $h = 0.5$: $\delta_1 = 1$, $\delta_2 = 0$. Node 2: secants 1, 0 differ, so $m_2 = 0$. Node 1 (end): $(3 \cdot 1 - 0)/2 = 1.5$, same sign as δ_1 : keep. Node 3 (end): $(3 \cdot 0 - 1)/2 = -0.5$, against $\delta_2 = 0$: set to 0.

21 nodes of $\min(k, 4)$: $k = 3.5$ is interior with secants 1, 1, so $m = 1$; $k = 4$: secants 1, 0, so $m = 0$.

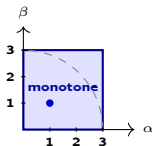
$\ln k$, 12 nodes ($h = 0.86$): secants 1.162, 0.568 give $m_1 = (3 \cdot 1.162 - 0.568)/2 = 1.459$, exactly SciPy's.

Fritsch & Carlson (1980); Fritsch & Butland (1984), the weights; end rule as in SciPy's PchipInterpolator. Why these rules: next frame. Ledger

pchip_power_form_3pt, spline_slope_system.

Where PCHIP's Formulas Come From

One piece, monotone. On $[x_i, x_{i+1}]$ with secant $\delta_i > 0$, measure the end slopes in units of the secant, $\alpha = m_i/\delta_i$ and $\beta = m_{i+1}/\delta_i$. The cubic rises throughout if $\alpha, \beta \geq 0$ and neither exceeds 3. A slope against the secant, or more than three times it, makes the cubic turn back inside the piece: an **overshoot**.



Interior node, two cases. Secants of opposite sign, or one zero: the data *turn* at x_i , a monotone curve is flat there, so $m_i = 0$. Secants of one sign: the size must stay inside the square for *both* neighbouring pieces, so take the **harmonic** mean, never more than twice the smaller secant; the weights w_1, w_2 lean toward the shorter interval.

Fritsch & Carlson (1980); Fritsch & Butland (1984); the end rule as in SciPy. Ledger `spline_slope_system`.

End node, a one-sided estimate. No secant to the left of x_1 , so fit the parabola through the first three points; by divided differences its slope at x_1 is

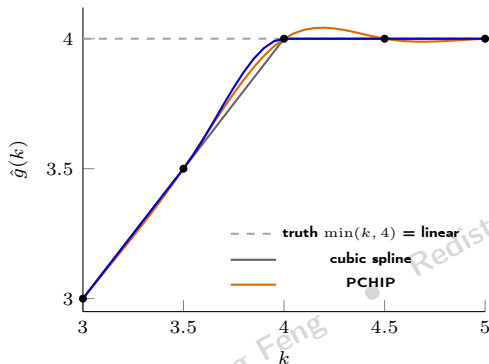
$$\begin{aligned} p'(x_1) &= \delta_1 + (x_1 - x_2) \frac{\delta_2 - \delta_1}{x_3 - x_1} \\ &= \frac{(2h_1 + h_2) \delta_1 - h_1 \delta_2}{h_1 + h_2}, \end{aligned}$$

on an even grid $(3\delta_1 - \delta_2)/2$. Then the two guards: 0 if it points against δ_1 ; capped at $3\delta_1$ if the data turn right after the start and it exceeds three times δ_1 .

Worked

$\ln k$ on 12 nodes, $h = 0.864$: $\delta_1 = 1.162$, $\delta_2 = 0.568$, so $m_1 = (3 \cdot 1.162 - 0.568)/2 = 1.459$, SciPy's value. $\min(k, 4)$: $\delta_1 = \delta_2 = 1$ gives $m_1 = 1$; at the right end $\delta = 0, 0$ gives $m_N = 0$.

PCHIP at the Kink: The Picture and the Arithmetic



Worked by hand, $h = 0.5$

$k = 3.5$: secants 1, 1, so $m = 1$. $k = 4$: secants 1, 0, so $m = 0$. On $[3.5, 4]$ the cubic with value 3.5 and slope 1 at the left end, value 4 and slope 0 at the right end, is $p = 3.5 + u + 2u^2 - 4u^3$ in $u = x - 3.5$, so $p(3.75) = 3.5 + 0.25 + 0.125 - 0.0625 = 3.8125$. The spline's system gives slope 0.5 at $k = 4$ instead: still rising at the cap, it reaches 4.043.

Why PCHIP cannot overshoot

A slope is 0 where the data turn and at most three times the smaller secant (twice on an even grid): Fritsch–Carlson's condition for a monotone cubic between two data values. Price: C^1 only, $O(h^3)$. Our formulas match SciPy to 9×10^{-16} .

$\min(k, 4)$, 21 nodes on $[0, 10]$, errors at 2,000 points; the kink sits on a node, so linear is exact.

Why PCHIP: What It Buys Over a Cubic Spline

What it buys

1. **It never overshoots the data** — monotone wherever the data are: 0 above the cap, against the spline's 0.043. That matters for policies, cdfs and Lorenz curves, and inside a fixed point, where an overshoot feeds back.
2. **It is local.** Change one data value and only the nearest pieces move: **4** of 20 intervals, against **all 20** for the natural spline. A kink stays where it is.
3. **No boundary condition to guess.** Its end slopes come from the data. On $\ln k$ the natural spline's $s'' = 0$ is wrong where $\ln k$ bends most: error 0.083, against PCHIP's 0.057.
4. **Still C^1 :** continuous slopes for Newton, EGM and Euler residuals, which linear lacks.

What it costs

C^1 , not C^2 : the curvature jumps at nodes. $O(h^3)$, not $O(h^4)$: on a smooth function a spline with a good boundary condition is more accurate — not-a-knot reaches 0.038 on $\ln k$. Slopes are set to 0 at a turning point, so peaks are flattened. And it keeps **monotonicity, not concavity**: for a concave $\hat{v}$ under golden section, use linear or Schumaker (next frame).

Choose PCHIP when shape and locality matter more than the last digit: constrained policies, distributions, kinked or noisy data. **Choose a spline** when the function is smooth and you can give it a sensible boundary.

Ledger why_pchip: locality on $\ln(1+k)$, 21 nodes; accuracy on $\ln k$, 12 nodes on $[0.5, 10]$, worst near $k = 0.8$.

Shape-Preserving Schemes: What Each One Guarantees

	accuracy	smooth	monotone	concavity
Linear	$O(h^2)$	C^0	yes	yes
Cubic spline	$O(h^4)$	C^2	no	no
PCHIP	$O(h^3)$	C^1	yes	no
(Fritsch–Carlson)				
Schumaker	$O(h^2)$	C^1	yes	yes

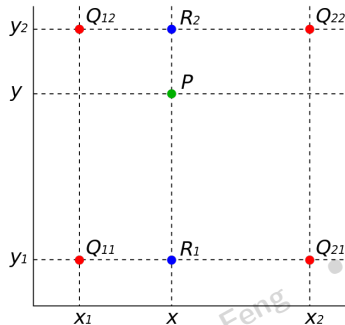
How to read the columns

Accuracy and smoothness are properties of the method. “Monotone” and “concavity” are **guarantees for every data set**: “no” means some data break it. On $\ln k$ all three schemes happened to stay concave; on the kinked policy the spline did not.

Schumaker’s quadratic spline is the only *smooth* (C^1) row that preserves both, which is why it is the standard choice when a value function must stay concave.

Fritsch & Carlson (1980); Schumaker (1983), *SIAM J. Numer. Anal.* 20(4); Judd & Solnick (1994).

Two Dimensions: Bilinear



Bilinear interpolation on a rectangular cell.

Bilinear: interpolate along x on both edges, then along y between the two results. Exact for $1, x, y, xy$; the natural extension of linear to a rectangular grid, and it keeps monotonicity in each argument separately.

The xy term is the catch. Along a cell's diagonal the interpolant is *quadratic*, so the surface bends inside every cell. A straight crease that cuts a cell diagonally — a constraint that binds when $a + z$ is small, say — cannot be represented; it is smeared over the whole cell.

And the cost: a tensor grid needs N^d points, which is 3.A4's curse arriving in the interpolation step rather than the maximization.

Triangular Interpolation, and Why It Handles a Kink

Split each cell into two triangles. Inside a triangle with vertices v_1, v_2, v_3 , write the point as a weighted average of the vertices (barycentric weights):

$$w_1 = \frac{(y_2 - y_3)(x - x_3) + (x_3 - x_2)(y - y_3)}{(y_2 - y_3)(x_1 - x_3) + (x_3 - x_2)(y_1 - y_3)},$$

w_2 likewise, $w_3 = 1 - w_1 - w_2$, and

$$\hat{f} = w_1 f_1 + w_2 f_2 + w_3 f_3.$$

- At a vertex one weight is 1; **inside, all lie in $(0, 1)$** — a convex combination, so it **cannot overshoot**. Outside, some weight is negative.
- The interpolant is **planar** on each triangle, so a crease that runs *along triangle edges* is reproduced exactly.

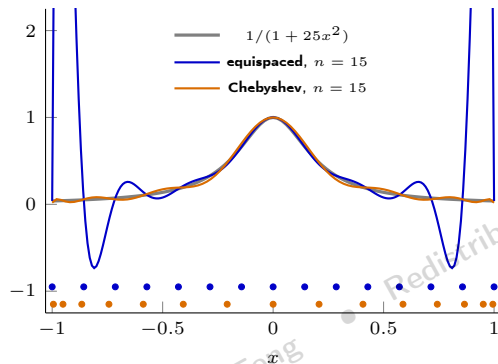
$\max(x + y - 1, 0)$: a crease on a diagonal

max error	$h = 0.25$	$h = 0.125$
bilinear	0.063	0.031
triangles along the crease	4×10^{-16}	4×10^{-16}
triangles across it	0.125	0.060
smooth f : bilinear	0.029	0.0077
smooth f : triangles	0.030	0.0077

Aligned with the kink, triangles are **exact**; across it, worse than bilinear; on a smooth function, the same. So split cells along the kink — or, on scattered points, put vertices on it (a Delaunay triangulation of an irregular grid).

Ledger tri_vs_bilinear; grid on $[0, 2]^2$, error at 401×401 points.

Global Polynomials, and Why Equally Spaced Nodes Fail



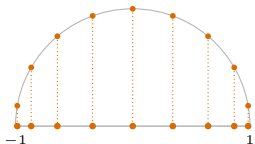
Dots at the bottom: the two node sets. The equispaced curve leaves the picture at the ends (it reaches 7.2).

Interpolate Runge's function on $[-1, 1]$ by a single polynomial through n nodes.

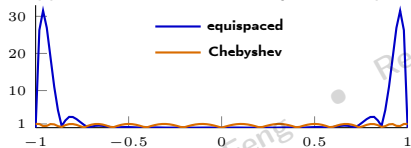
n	equispaced	Chebyshev
5	0.44	0.40
9	1.05	0.17
15	7.19	0.047
21	59.8	0.015

More points makes the equispaced fit **worse**, by a factor of 136 between $n = 5$ and $n = 21$. The oscillation is at the ends, and it diverges. Same function, same degree, nodes moved: the Chebyshev fit converges.

Why Clustering at the Ends Works



Chebyshev nodes: **equally spaced angles** on a half circle, dropped onto the axis — they bunch up near ± 1 .



$|\prod_i (x - x_i)|$, $n = 15$, in units of 2^{1-n} .

Ledger fig_node_poly. Error formula: any numerical-analysis text, e.g. Judd (1998), ch. 6.

The interpolation error of a polynomial through n nodes is

$$f(x) - p(x) = \frac{f^{(n)}(\xi)}{n!} \prod_{i=1}^n (x - x_i).$$

The first factor belongs to f . The second belongs to **you** — it depends only on where you put the nodes.

Intuition. A polynomial is stiff: pinned at the nodes, it swings hardest where it is least pinned. Equally spaced nodes leave the ends loosely held, and $|\prod (x - x_i)|$ there is **31 times** Chebyshev's maximum, while in the middle it is **35 times smaller**. Chebyshev nodes move points to the ends until every ripple has the same height — the **smallest possible maximum**, 2^{1-n} .

Chebyshev Nodes: Cluster at the Ends

On $[-1, 1]$,

$$x_i = \cos\left(\frac{2i-1}{2n}\pi\right), \quad i = 1, \dots, n,$$

mapped to $[a, b]$ by $s_i = \frac{a+b}{2} + \frac{b-a}{2}x_i$. These nodes **minimise the maximum** of the node polynomial

$\prod_i (x - x_i)$, which makes the interpolation error near the best possible, and they are dense exactly where the equispaced fit blew up — at the boundaries.

At $n = 21$ the Chebyshev error is 0.015 against the equispaced 59.8: **four thousand times better**, with the same number of points and the same degree.

Read the formula carefully

$i = 1$ gives $\cos(\pi/2n) \approx +1$: the nodes come out **descending**, while every routine downstream — a bracketing search, `np.interp` — assumes an ascending grid. **Sort them.**

A second set: the extrema

The m points where T_{m-1} reaches ± 1 , $x_j = -\cos(\pi(j-1)/(m-1))$, $j = 1, \dots, m$, also cluster at the ends but **include** ± 1 and come out ascending. With $m = 3, 5, 9, 17, \dots$ each set **contains the one before**; the zeros do not. Smolyak, next, is built on the extrema.

The Curse of Dimensionality: A Full Tensor Grid

In one dimension we place m nodes. In d dimensions the obvious grid takes **every combination** of them — the tensor product

$$G \times G \times \cdots \times G \quad (d \text{ times}),$$

with m^d points.

Each state variable you add — capital, a shock, an aggregate moment, a second country — **multiplies** the count by m .

And most of those points sit where several coordinates are extreme at once: the corners and edges of the box, states the model almost never visits.

Ledger smolyak, columns tensor_3/5/9_per_dim.

d	3^d	5^d	9^d
2	9	25	81
3	27	125	729
4	81	625	6,561
5	243	3,125	59,049
10	59,049	9.8 million	3.5×10^9
12	531,441	244 million	2.8×10^{11}

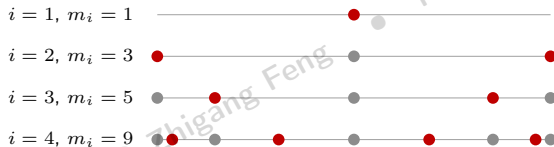
At ten dimensions, five nodes per axis is already ten million function values — each perhaps a model solve. The next frames build Smolyak's grid, which keeps a few hundred of them.

The Ladder: Level i Counts Refinements, m_i Counts Nodes

Smolyak refines each direction in steps called **levels**. The level i is a label; m_i is how many nodes that level has:

level i	1	2	3	4	5
nodes m_i	1	3	5	9	17

$m_1 = 1$ (the centre) and $m_i = 2^{i-1} + 1$: each level roughly doubles the count. The nodes are the Chebyshev **extrema**, so each level keeps every node of the one below and adds new ones in the gaps:



red: new at this level; grey: inherited.

A bigger grid means a higher level

To make the grid finer in a direction you raise its level i , and its node count m_i roughly doubles. A tensor grid at level i in every direction has m_i^d points: level 3 everywhere is 5^d , level 4 is 9^d — the columns of the previous table.

Why nesting matters. Pieces built from different levels overlap, so their union has few points; and raising the budget (next frame) only *adds* points — every function value already paid for is reused.

μ : A Budget on Total Refinement

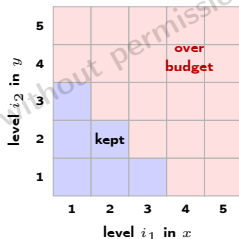
Every direction starts at level 1: a single node, the centre.
 Raising direction j to level i_j spends $i_j - 1$ steps of refinement. Smolyak hands you μ steps for all directions together:

$$(i_1 - 1) + (i_2 - 1) + \cdots + (i_d - 1) \leq \mu.$$

Why a budget. A piece $G^{i_1} \times \cdots \times G^{i_d}$ has $m_{i_1} \cdots m_{i_d}$ points, and m roughly doubles per level. Refining every direction at once multiplies those doublings — that is the tensor grid. Capping the *sum* of the refinements caps the *product*.

With $\mu = 2$ in two dimensions you may spend both steps on x , $(3, 1)$; both on y , $(1, 3)$; or one on each, $(2, 2)$ — never level 3 in both (the blue cells on the right).

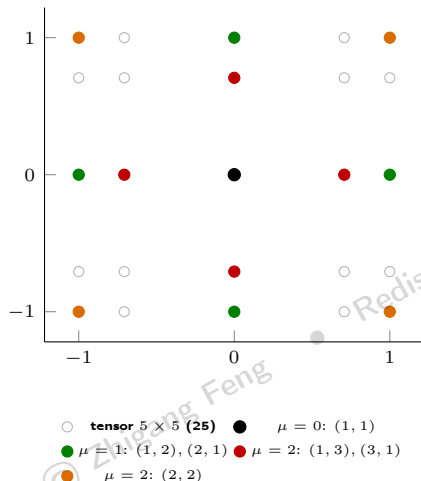
How large to make μ is a choice, measured in “Choosing μ ” below.



μ	$d = 2$		$d = 4$	
	Smolyak	tensor	Smolyak	tensor
1	5	9	9	81
2	13	25	41	625
3	29	81	137	6,561
4	65	289	401	83,521

Points; tensor = level $\mu + 1$ in every direction.

$q = d + \mu$, and Building It in Two Dimensions, By Hand



Count in levels instead of steps:

$$\sum_j (i_j - 1) \leq \mu \iff |i| = \sum_j i_j \leq d + \mu \equiv q.$$

q is the same budget plus the d free level-1 starts.
 The grid is the union of the small tensor grids of every admissible i :

$$\mathcal{G} = \bigcup_{|i| \leq q} G^{i_1} \times \cdots \times G^{i_d}.$$

In 2-D, $q = 2 + \mu$:

$\mu = 0$: only (1, 1) — the centre. **One point.**

$\mu = 1$: (1, 2), (2, 1) add the ends of each axis — the five-point **cross**.

$\mu = 2$: (1, 3), (3, 1) refine each axis to level 3; (2, 2) adds the **four corners**. **13** of the 25 tensor points. The 12 dropped, like $(\pm 0.71, \pm 0.71)$, are fine in *both* coordinates at once.

Why the Sum Starts at $q - d + 1$

The literature writes the grid with a lower bound,

$$\mathcal{G}(q, d) = \bigcup_{q-d+1 \leq |i| \leq q} G^{i_1} \times \cdots \times G^{i_d}.$$

For the grid it changes nothing. By nesting, a piece below the band sits inside one in it: raise one coordinate of i until $|i| = q$, and $G^i \subset G^{i'}$. Measured: the top layer $|i| = q$ alone, the band, and all $|i| \leq q$ give the same set for every (d, μ) we tried — 13 points each at $d = 2$, $\mu = 2$.

It is there for the interpolant. Smolyak's approximation adds tensor interpolants on the pieces. The pieces overlap, so the sum counts the overlaps too often, and the correction goes exactly $d - 1$ layers down. **The lower bound drops exactly the terms that carry no weight.**

Ledger smolyak_layers (each grid counted three ways) and smolyak.index_sets_2d_mu2.

Inclusion-exclusion in 2-D, $\mu = 2$

Top pieces $A = (1, 3)$, $B = (2, 2)$, $C = (3, 1)$.
Overlaps: $A \cap B = (1, 2)$, $B \cap C = (2, 1)$, $A \cap C = A \cap B \cap C = (1, 1)$. So

$$A + B + C - (1, 2) - (2, 1) - \underbrace{(1, 1) + (1, 1)}_{=0}.$$

i	$ i $	weight $(-1)^{q- i } \binom{d-1}{q- i }$
$(1, 1)$	2	0
$(1, 2), (2, 1)$	3	-1
$(1, 3), (2, 2), (3, 1)$	4	+1

The weight is zero once $q - |i| > d - 1$, that is $|i| < q - d + 1$.

Three Dimensions, and the Signed Combination

At $d = 3$, $\mu = 2$ ($q = 5$) the bound $q - d + 1 = 3$ excludes nothing. The ten admissible indices, by layer:

$ i $	indices	weight
5	(1, 1, 3) (1, 3, 1) (3, 1, 1)	+1
	(1, 2, 2) (2, 1, 2) (2, 2, 1)	
4	(1, 1, 2) (1, 2, 1) (2, 1, 1)	-2
3	(1, 1, 1)	+1

Their union is **25 distinct points**, against $5^3 = 125$.

The approximation is *not* a plain interpolation on those points. It is the signed combination of tensor interpolants,

$$\hat{f} = \sum_{q-d+1 \leq |i| \leq q} (-1)^{q-|i|} \binom{d-1}{q-|i|} (U^{i_1} \otimes \dots \otimes U^{i_d}),$$

which cancels the overlaps the union created.

Check: the weights sum to $6 - 6 + 1 = 1$ (in 2-D, $3 - 2 + 0 = 1$), so a constant is reproduced exactly, as it must be.

Signed weights: a sparse *quadrature* rule built this way can have negative ones, so it need not be positive — unlike Gauss rules (4.B2).

Choosing μ : Points Against Accuracy

μ is **your choice**. Raising it adds points, and how much accuracy that buys depends on the function. Interpolate on $[-1, 1]^d$ and measure the error at 5,000 random points:

μ	points	$d = 2$		$d = 4$		
		smooth	kinked	points	smooth	
1	5	0.37	0.96	9	1.2	1.6
2	13	0.066	0.33	41	0.44	1.2
3	29	0.007	0.33	137	0.095	0.65
4	65	0.0006	0.17	401	0.027	0.65

smooth: $e^{-\|x\|^2}$; kinked: $\max(x_1 + \dots + x_d, 0)$.

The trade-off, and the error

The error bound $C_d n^{-k} (\log n)^{(d-1)(k+1)}$ says it: μ raises the number of points n ; the **smoothness** k decides what each extra point is worth. Smooth: about ten times less error per step in 2-D. Kinked: the error barely moves.

How to choose

Three criteria: the accuracy you need, measured **off** the grid; the **cost of one point** (each may be a model solve); and the **smoothness** of f . Start low and raise μ until the off-grid error meets your target or stops falling — each step reuses every point already computed. If it stalls, more μ is the wrong tool: go local or split the domain.

What Sparsity Buys, and What It Costs

d	Smolyak $\mu=2$	tensor 5^d
2	13	25
3	25	125
4	41	625
5	61	3,125
10	221	9.8 million
12	313	244 million

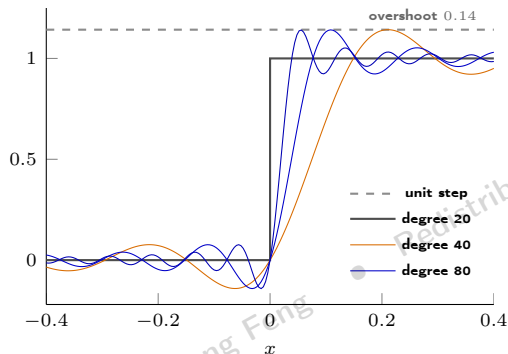
Buys: at ten dimensions the sparse grid needs **221 points** where a five-point tensor grid needs nearly ten million — a factor of **44,000**.

Costs: think of f as main effects plus interactions. The kept cells of the index square resolve x finely *or* y finely; the dropped corner resolves both at once — the high-order **interaction** terms, $x^8 y^8$ rather than x^8 and y^8 .

Cheap when interactions are small: smooth value functions, where the error kept falling. **Expensive when** the feature *is* an interaction — a kink along $x + y$, a borrowing constraint that binds only when several states are low together: the kinked column stalled. The bound says it in one line: the tensor rate n^{-k} , times a log penalty growing with d . **The curse is weakened, not broken.**

Here Smolyak is a **node set**. In Session 5 the same index rule picks a **basis** for projection — same combinatorics, different job.

Where Polynomials Cannot Be Rescued: Gibbs



Runge's problem is fixed by moving the nodes; a genuine **jump** is not. Interpolating a unit step at Chebyshev nodes, every curve **overshoots** by 0.14 — 0.142 and 0.141 at degrees 40 and 80. Higher degree squeezes the ringing towards the jump (peak at $x = 0.21, 0.11, 0.05$) **but not lower**.

Why. A sum of smooth global functions cannot turn a corner: to climb by 1 over a short distance it must climb past the top before it bends back. A steeper climb only narrows the wiggle.

What to do instead

Go **local**, or split the domain at the kink. Economics produces kinks routinely: borrowing constraints, bounds, discrete choices; Smolyak inherits the problem. (ReLU networks, with a kinked basis, cope better — S7.)

Takeaways

- Interpolation is not a detail of a dynamic program — it is inside the fixed point, so its error **compounds through every sweep**.
- **Linear interpolation preserves concavity**, and that is precisely what licenses golden section inside the Bellman loop.
- Cubic splines buy $O(h^4)$ and lose shape: on a capped policy the spline **overshot the cap by 0.043**. PCHIP's local slopes never overshoot; in 2-D, triangles aligned with a kink reproduce it exactly.
- Equally spaced nodes make a global polynomial **worse** as you add points — error 0.44 to 59.8 from $n = 5$ to 21. Chebyshev nodes fix it; at $n = 21$ they are four thousand times better.
- $x_i = \cos((2i - 1)\pi/2n)$ comes out **descending**. Sort it.
- A tensor grid in many dimensions is mostly waste. **Smolyak** gives all directions one refinement budget, $|i| \leq q = d + \mu$, on nested Chebyshev extrema: 221 points in ten dimensions against 9.8 million. It returns in Session 5 as a *basis*.
- A discontinuity cannot be fixed by moving nodes. **Gibbs does not go away** — go local, or split the domain.

Bridge: The Other Thing a Grid Cannot Do

We can now represent $\hat{v}$ between grid points and know which schemes keep the properties the algorithms need.

But the Bellman equation does not only evaluate $\hat{v}$ — it takes an **expectation** of it:

$$\mathbb{E}[\hat{v}(k', z') \mid z] = \int \hat{v}(k', z') dF(z' \mid z).$$

Every solve so far has replaced that integral with a finite sum $\sum_m \Pi_{jm} \hat{v}(k', z_m)$ — and we have written Π down by hand twice now, without ever saying where it comes from.

Next (4.B2)

Expectations. First quadrature: how an integral becomes a weighted sum of a few well-chosen points, and why a rule that knows the density needs so few of them.

Then, in the same deck, where Π comes from when the shock is persistent — Tauchen and Rouwenhorst, and which of the two you should actually use.